

数学知识补充

莫比乌斯

莫比乌斯反演

随机化

杂项

大数质因数分解（到达 $1e18$ ）

计算几何部分知识

三角函数

一、基础三角函数

1. 正弦（`sin`）

2. 余弦（`cos`）

3. 正切（`tan`）

二、反三角函数（角度计算核心）

1. 反余弦（`acos`）

2. 四象限反正切（`atan2`）

3. 反正切（`atan`）

三、竞赛注意事项

向量

二、核心性质

1. 反交换律

2. 不满足结合律

3. 与数乘的分配律

4. 与加法的分配律

5. 自身叉积为零向量

6. 模长的几何意义

7. 二维向量的特殊情况

8. 与点积的关系（拉格朗日恒等式）

三、典型应用

函数及其他

数学知识补充

莫比乌斯

$$\mu(n) = \begin{cases} 1 & n = 1 \\ (-1)^k & n = p_1 p_2 \cdots p_k \\ 0 & \text{其余情况} \end{cases}$$

```
#include<cstdio>
const int N = 1e6 + 5;
int mu[N], vis[N], prime[N];
int tot; //用来记录prime的个数
void init(){
    mu[1] = 1;
    for(int i = 2; i < N; i++){
        if(!vis[i]){
            prime[tot++] = i;
            mu[i] = -1;
        }
        for(int j = 0; j < tot && i * prime[j] < N; j++){
            vis[i * prime[j]] = 1;
            if(i % prime[j]) mu[i * prime[j]] = -mu[i];
            else{
                mu[i * prime[j]] = 0;
                break;
            }
        }
    }
}
int main(){
    init();
}
```

- 性质一：积性函数
- 性质二：

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & (n = 1) \\ 0 & (n > 1) \end{cases}$$

- 性质三：

$$\sum_{i=1}^n \sum_{d|i} \mu(d) = \sum_{d=1}^n \left\lfloor \frac{n}{d} \right\rfloor \mu(d)$$

- 性质四： 其中分子为欧拉函数

$$\sum_{d|n} \frac{\mu(d)}{d} = \frac{\varphi(n)}{n}$$

- 经典应用一： $n < m$ ，使用整除分块可优化

$$\sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = 1] = \sum_{d=1}^{\min(n, m)} \mu(d) \cdot \left\lfloor \frac{n}{d} \right\rfloor \cdot \left\lfloor \frac{m}{d} \right\rfloor$$

- 应用二：

$$f(x) = \sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = x] \quad (n \leq m)$$

$$\sum_{i=1}^{\left\lfloor \frac{n}{x} \right\rfloor} \sum_{j=1}^{\left\lfloor \frac{m}{x} \right\rfloor} [\gcd(i, j) = 1] = \sum_{d=1}^{\min(\left\lfloor \frac{n}{x} \right\rfloor, \left\lfloor \frac{m}{x} \right\rfloor)} \mu(d) \cdot \left\lfloor \frac{n}{x \cdot d} \right\rfloor \cdot \left\lfloor \frac{m}{x \cdot d} \right\rfloor$$

莫比乌斯反演

常用公式

$$\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{d=1}^n \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{n}{d} \right\rfloor$$

$$f(n) = \sum_{d|n} \mu(d) \cdot F\left(\frac{n}{d}\right) = \sum_{d|n} \mu\left(\frac{n}{d}\right) \cdot F(d) \quad (d \text{ 是 } n \text{ 的约数})$$

$$f(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) F(d) \quad (d \text{ 是 } n \text{ 的倍数})$$

对于一些函数 $f(n)$ ，如果我们很难求出其数值，却很容易求出其约数和倍数和 $F(n)$ ，那么可以通过莫比乌斯反演得到 $f(n)$ 的数值。

莫比乌斯反演定理：

对于待求解函数 $f(n)$

第一步：先构造另一个约数和（或倍数和）函数 $F(n)$ ，它满足

$$F(n) = \sum_{d|n} f(d) \quad \text{或者} \quad F(n) = \sum_{n|d} f(d)$$

第二步：计算 $F(n)$ 和莫比乌斯函数 $\mu(n)$

第三步：通过 $F(n)$ 、莫比乌斯函数 $\mu()$ 来求解 $f(n)$

$$f(n) = \sum_{d|n} \mu(d) \cdot F\left(\frac{n}{d}\right) \quad \text{或者} \quad f(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) \cdot F(d)$$

随机化

- `std::mt19937`: 32 位 Mersenne Twister 算法，高性能、高周期（ $2^{19937}-1$ ），最常用。
- `std::mt19937_64`: 64 位版本的 Mersenne Twister，适用于需要 64 位随机数的场景。
- `std::minstd_rand0`/`std::minstd_rand`: 线性同余生成器，兼容性好但周期较短。
- `std::random_device`: 硬件随机数生成器（如系统提供的真随机源），通常用于为其他生成器播种。

```
#include <iostream>
#include <random> // 随机数库
#include <chrono> // 用于获取时间作为种子
int main() {
    // 1. 初始化随机数生成器（引擎）
    // 用当前时间作为种子，确保每次运行结果不同
    unsigned seed =
std::chrono::system_clock::now().time_since_epoch().count();
    std::mt19937 rng(seed); // Mersenne Twister生成器
    // 2. 定义分布（例如：生成[1, 100]之间的整数）
    std::uniform_int_distribution<int> dist(1, 100);
    // 3. 生成随机数
    for (int i = 0; i < 5; ++i) {
        std::cout << dist(rng) << " "; // 用生成器驱动分布
    }
    // 输出示例：37 89 12 5 76（每次运行不同）
    return 0;
}
```

```
//伪随机数生成
uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
}
```

杂项

对于任意值 a, b , 有

$$|a + b| + |a - b| = 2 \max(|a|, |b|)$$

通过坐标变换 $u = x + y$ 和 $v = x - y$, 切比雪夫距离可以转换为曼哈顿距离的一半:

$$\max(|x_1 - x_2|, |y_1 - y_2|) = \frac{|u_1 - u_2| + |v_1 - v_2|}{2}$$

对任意数组 $(A[1..n])$, 有: $[\gcd(A_1, A_2, \dots, A_n) = \gcd(A_1, A_2 - A_1, \dots, A_n - A_{n-1})]$

$$\gcd(x, y, z, \dots) = \gcd(x, y - x, z - x, \dots)$$

$$\text{对于异或操作有, } a - b \leq a \oplus b \leq a + b$$

大数质因数分解（到达1e18）

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <algorithm>
#include <cmath>
using namespace std;

// 快速计算 gcd (欧几里得算法)
long long gcd(long long a, long long b) {
    while (b) {
```

```

        a %= b;
        swap(a, b);
    }
    return a;
}

// 使用 __int128 的模乘 (防止 1e18 乘法溢出)
long long mod_mul(long long a, long long b, long long mod) {
    return (long long)((__int128)a * b % mod);
}

// 快速幂取模 (基于 mod_mul)
long long mod_exp(long long base, long long exp, long long mod) {
    long long res = 1;
    while (exp > 0) {
        if (exp & 1) res = mod_mul(res, base, mod);
        base = mod_mul(base, base, mod);
        exp >>= 1;
    }
    return res;
}

// Miller-Rabin 素性测试 (确定性基, 覆盖 64 位整数)
bool is_prime(long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    // 分解 n-1 = d * 2^s
    long long d = n - 1;
    int s = 0;
    while (d % 2 == 0) {
        d /= 2;
        s++;
    }

    vector<long long> bases;
    if (n < 2047) bases = {2};
    else if (n < 9080191) bases = {31, 73};
    else if (n < 4759123141LL) bases = {2, 7, 61};
    else if (n < 1122004669633LL) bases = {2, 13, 23, 1662803};
    else if (n < 2152302898747LL) bases = {2, 3, 5, 7, 11};
    else if (n < 3474749660383LL) bases = {2, 3, 5, 7, 11, 13};

```

```

        else if (n < 341550071728321LL) bases = {2, 3, 5, 7, 11, 13,
17};
        else bases = {2, 325, 9375, 28178, 450775, 9780504,
1795265022};

        for (long long a : bases) {
            if (a % n == 0) continue; // 跳过 a 是 n 倍数的情况
            long long x = mod_exp(a, d, n);
            if (x == 1 || x == n - 1) continue;
            bool composite = true;
            for (int r = 1; r < s; r++) {
                x = mod_mul(x, x, n);
                if (x == n - 1) {
                    composite = false;
                    break;
                }
            }
            if (composite) return false;
        }
        return true;
    }

// Pollard's Rho 核心算法 (Floyd 判圈法)
long long pollard_rho(long long n) {
    if (n % 2 == 0) return 2; // 偶数快速处理
    long long x = rand() % (n - 2) + 2;
    long long y = x;
    long long c = rand() % (n - 1) + 1;
    long long d = 1;

    auto f = [&](long long x) {
        return (mod_mul(x, x, n) + c) % n;
    };

    while (d == 1) {
        x = f(x);
        y = f(f(y));
        d = gcd(abs(x - y), n);
        if (d == n) break; // 失败, 需调整参数
    }
    return d;
}

```

```

// 完整质因数分解 (小质数预处理 + Pollard's Rho)
vector<long long> factorize(long long n) {
    vector<long long> factors;

    // 步骤 1: 小质数试除预处理 (到 10^6)
    for (long long i = 2; i * i <= n && i <= 1000000; i++) {
        while (n % i == 0) {
            factors.push_back(i);
            n /= i;
        }
    }
    if (n == 1) return factors;

    // 步骤 2: 检查剩余部分是否为质数
    if (is_prime(n)) {
        factors.push_back(n);
        return factors;
    }

    // 步骤 3: Pollard's Rho 分解剩余合数
    long long d = n;
    while (d == n || d == 1) {
        d = pollard_rho(n);
    }

    vector<long long> fac1 = factorize(d);
    vector<long long> fac2 = factorize(n / d);
    factors.insert(factors.end(), fac1.begin(), fac1.end());
    factors.insert(factors.end(), fac2.begin(), fac2.end());

    return factors;
}

vector<long long> prime_factors(long long n) {
    vector<long long> factors = factorize(n);
    sort(factors.begin(), factors.end());
    return factors;
}

int main() {
    srand(time(0)); // 初始化随机数种子 (重要!)
    int t;
    cin >> t;

```



```
long long n1 = 999999937LL * 999999929LL; // ≈ 1e18
vector<long long> factors1 = prime_factors(t);
cout << "Factors of " << t << ":\n";
for (long long p : factors1) cout << p << " ";
cout << "\n";

return 0;
}
```

计算几何部分知识

三角函数

一、基础三角函数

1. 正弦 (`sin`)

- 数学定义：直角三角形中对边与斜边的比值 ($\sin\theta = \text{对边}/\text{斜边}$)
- 竞赛核心用途
 - 向量的垂直分量计算： $y = r * \sin\theta$ (r 为向量模长， θ 为与 x 轴夹角)
 - 三角形面积公式： $\text{面积} = 0.5 * a * b * \sin\theta$ (a, b 为两边长度， θ 为夹角)
 - 图形旋转时的坐标变换 (结合 `cos` 使用)

2. 余弦 (`cos`)

- 数学定义：直角三角形中邻边与斜边的比值 ($\cos\theta = \text{邻边}/\text{斜边}$)
 - 竞赛核心用途
- :

- 向量的水平分量计算: $x = r \cdot \cos\theta$
- 向量点积公式: $a \cdot b = |a| |b| \cos\theta$ (用于判断夹角大小、向量垂直性)
- 计算两点距离的平方 (间接使用, 避免开方损失精度)

3. 正切 (`tan`)

- 数学定义: $\tan\theta = \sin\theta / \cos\theta$ (对边与邻边的比值)
- 竞赛核心用途:
 - 直线斜率计算: $k = \tan\theta = \Delta y / \Delta x$ (θ 为直线与 x 轴夹角)
 - 快速比较角度大小 (避免复杂的角度转换)

二、反三角函数 (角度计算核心)

1. 反余弦 (`acos`)

- 功能: 输入余弦值, 返回对应的角度 (范围 $[0, \pi]$, 弧度制)
- 竞赛核心用途
 - 计算两向量夹角: $\theta = \arccos((a \cdot b) / (|a| \cdot |b|))$ (点积公式逆用)
 - 求圆周率: `const double pi = acos(-1.0)` (标准写法, 精度可靠)

2. 四象限反正切 (`atan2`)

- 功能: 输入 (y, x) , 返回点 (x, y) 与原点连线和 x 轴的夹角 (范围 $[-\pi, \pi]$, 自动区分象限)
- 竞赛核心用途
(最常用反三角函数):
 - 向量方向角计算: 直接通过向量坐标 (x, y) 得到角度 (无需手动判断象限)
 - 极坐标转换: 将直角坐标 (x, y) 转为极角 θ
 - 排序向量: 按角度大小对向量排序 (如凸包问题中的极角排序)

3. 反正切 (`atan`)

- 功能：输入正切值 `k`，返回角度（范围 $[-\pi/2, \pi/2]$ ）
- 竞赛限制：无法区分 `(x,y)` 和 `(-x,-y)` 所在象限，仅在已知象限信息时使用

三、竞赛注意事项

1. 弧度制：C++ 标准库函数（`sin/cos` 等）均使用弧度制，角度需转换（ $\text{角度}^\circ * \pi / 180$ ）

向量

1. 多边形自身的逆时针序：指多边形的顶点按顺序排列时，沿着顶点走会形成逆时针方向的闭合图形（比如正方形的四个顶点按 $(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (1,0) \rightarrow (0,0)$ 排列，是逆时针的）。这是多边形“自身形状”的属性，与外部点 `P` 无关。
2. 向量的转向：代码中 `a[i]` 是“顶点 - `P`”的向量（从 `P` 指向多边形顶点）。相邻向量 `a[i]` 和 `a[i+1]` 的转向（顺时针 / 逆时针），取决于 `P` 的位置，与多边形自身的顶点顺序无关。

一、基本定义（三维向量）

对于三维向量 $\mathbf{a} = (a_1, a_2, a_3)$ 和 $\mathbf{b} = (b_1, b_2, b_3)$ ，叉积定义为：

$$\mathbf{a} \times \mathbf{b} = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$$

其结果是一个新向量，而非标量，且满足：

- 方向：垂直于 `a` 和 `b` 所确定的平面，遵循右手螺旋定则（右手四指从 `a` 转向 `b`，拇指指向即为 `a × b` 的方向）。
- 模长： $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| \cdot |\mathbf{b}| \cdot \sin\theta$ （ θ 为 `a` 与 `b` 的夹角， $0 \leq \theta \leq \pi$ ），数值上等于 `a` 和 `b` 构成的平行四边形的面积。

二、核心性质

1. 反交换律

$$\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$$

- 几何意义： $\mathbf{a} \times \mathbf{b}$ 与 $\mathbf{b} \times \mathbf{a}$ 大小相等、方向相反（右手螺旋方向相反）。

2. 不满足结合律

$$(\mathbf{a} \times \mathbf{b}) \times \mathbf{c} \neq \mathbf{a} \times (\mathbf{b} \times \mathbf{c})$$

- 说明：叉积的结合顺序会改变结果，需严格按顺序计算。

3. 与数乘的分配律

对于任意实数 k ：

$$k(\mathbf{a} \times \mathbf{b}) = (k\mathbf{a}) \times \mathbf{b} = \mathbf{a} \times (k\mathbf{b})$$

- 几何意义：数乘对叉积的缩放作用与对向量的缩放一致。

4. 与加法的分配律

$$\mathbf{a} \times (\mathbf{b} + \mathbf{c}) = \mathbf{a} \times \mathbf{b} + \mathbf{a} \times \mathbf{c}$$

$$(\mathbf{a} + \mathbf{b}) \times \mathbf{c} = \mathbf{a} \times \mathbf{c} + \mathbf{b} \times \mathbf{c}$$

- 说明：叉积对向量加法满足分配律，可展开计算。

5. 自身叉积为零向量

$$\mathbf{a} \times \mathbf{a} = \mathbf{0} \text{ (零向量)}$$

- 推导：当 $\mathbf{a}$ 与 $\mathbf{b}$ 同向 ($\theta = 0$) 或反向 ($\theta = \pi$) 时， $\sin\theta = 0$ ，故模长为 0，即零向量。

- 应用：判断两向量是否共线（ $\mathbf{a} \times \mathbf{b} = \mathbf{0} \Leftrightarrow \mathbf{a}$ 与 $\mathbf{b}$ 共线）。

6. 模长的几何意义

$$|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| \cdot |\mathbf{b}| \cdot \sin\theta$$

- 当 $\mathbf{a}$ 与 $\mathbf{b}$ 垂直（ $\theta = \pi/2$ ）时， $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| \cdot |\mathbf{b}|$ （模长等于两向量模长的乘积）。
- 数值上等于 $\mathbf{a}$ 和 $\mathbf{b}$ 构成的平行四边形的面积（可推广为三角形面积的 2 倍）。

7. 二维向量的特殊情况

二维向量可视为 z 分量为 0 的三维向量，其叉积结果的 z 分量为 $a_1b_2 - a_2b_1$ ，其他分量为 0。因此，二维叉积通常简化为标量：

$$\text{cross}(\mathbf{a}, \mathbf{b}) = a_1b_2 - a_2b_1$$

- 符号意义：
 - 若 > 0 ： $\mathbf{b}$ 在 $\mathbf{a}$ 的逆时针方向；
 - 若 < 0 ： $\mathbf{b}$ 在 $\mathbf{a}$ 的顺时针方向；
 - 若 $= 0$ ： $\mathbf{a}$ 与 $\mathbf{b}$ 共线。

8. 与点积的关系（拉格朗日恒等式）

$$|\mathbf{a} \times \mathbf{b}|^2 + (\mathbf{a} \cdot \mathbf{b})^2 = |\mathbf{a}|^2 \cdot |\mathbf{b}|^2$$

- 推导：由 $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| |\mathbf{b}| \sin\theta$ 和 $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos\theta$ ，结合 $\sin^2\theta + \cos^2\theta = 1$ 可得。

三、典型应用

1. 判断向量共线： $\mathbf{a} \times \mathbf{b} = \mathbf{0} \Leftrightarrow \mathbf{a}$ 与 $\mathbf{b}$ 共线。
2. 计算平行四边形 / 三角形面积： $S = |\mathbf{a} \times \mathbf{b}|$ （平行四边形）或 $S = |\mathbf{a} \times \mathbf{b}| / 2$ （三角形）。

3. 判断平面中三点的转向：二维叉积的符号可判断三点构成的角是顺时针、逆时针还是共线（如多边形顶点顺序判断）。
4. 求法向量：三维叉积 $\mathbf{a} \times \mathbf{b}$ 是 $\mathbf{a}$ 和 $\mathbf{b}$ 所在平面的法向量，用于平面方程求解。
5. 物理中力矩、角动量的计算：力矩 $\mathbf{M} = \mathbf{r} \times \mathbf{F}$ 、角动量 $\mathbf{L} = \mathbf{r} \times \mathbf{p}$ 均依赖叉积定义。

函数及其他

`std::next_permutation` 会直接将一个序列变换为它的下一个字典序排列。如果当前序列已经是最后一个排列，它会返回 `false`。

```
#include <iostream>
#include <vector>
#include <algorithm> // 包含 std::next_permutation

// 打印一个整数向量
void printVector(const std::vector<int>& vec) {
    for (int num : vec) {
        std::cout << num << " ";
    }
    std::cout << std::endl;
}

int main() {
    int x;
    std::cout << "请输入一个非负整数 x: ";
    std::cin >> x;

    // 处理输入无效的情况
    if (x < 0) {
        std::cerr << "错误：输入必须是非负整数。" << std::endl;
        return 1; // 返回非零表示错误
    }

    // 如果 x 为 0，则没有元素，不输出任何排列
    if (x == 0) {
        return 0;
    }
}
```

```
}

// 1. 创建并初始化一个包含 1 到 x 的向量
std::vector<int> numbers;
for (int i = 1; i <= x; ++i) {
    numbers.push_back(i);
}

// 2. 循环生成并打印所有排列
// 使用 do-while 是因为第一个排列（初始状态）也需要被打印
do {
    printVector(numbers);
} while (std::next_permutation(numbers.begin(),
numbers.end()));

return 0;
}
```